

SDEV2171

Lesson 19: Authentication Concepts

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, students should be able to:

- explain the purpose of authentication in a mobile app
- describe the order of registration, login, logout, and one session-validation example
- identify how auth state changes route access
- distinguish public and protected routes in an Expo Router app
- explain how a bearer token reaches one protected request

WE ARE ESSENTIAL TO ALBERTA | NAIT

Lesson Roadmap

1. Reconnect lesson 18 to a new auth-driven navigation problem
2. Define the core auth terms
3. Map the auth flow and route split
4. Inspect one shared auth state provider
5. Inspect one session-validation request and one bearer-token request
6. Checkpoint and lesson-20 handoff

WE ARE ESSENTIAL TO ALBERTA | NAIT

Starting Point From Lesson 18

- keep the same study-plan example line
- keep the same `{ id, label }` study-step shape
- keep the same shared selected-step workflow
- keep the same local draft form flow

What changes today

- some screens must become public
- some screens must become protected
- the app needs shared auth state
- one token must be attached to a protected request

WE ARE ESSENTIAL TO ALBERTA | NAIT

Why We Switch From Expo Go

- Expo Go is still useful for quick UI checks
- Expo documents development builds as the path that adds stronger tooling such as network request inspection
- auth work is easier when you can inspect login, registration, and session-validation requests directly
- lesson 11 mostly needed logs and LogBox, but module 05 benefits from a development build workflow

WE ARE ESSENTIAL TO ALBERTA | NAIT

Why Authentication Exists

- some app features should only work for a signed-in user
- some backend data should not be public
- some navigation paths depend on who the user is

Plain-language version

- auth answers: “Who are you, and what can you access?”

WE ARE ESSENTIAL TO ALBERTA | NAIT

Core Terms

- `register` : create an account
- `login` : prove identity and receive a token
- `logout` : clear auth access and leave the protected area
- `token` : the proof the app sends on later protected requests
- `session validation` : check that the current token still represents a valid user

WE ARE ESSENTIAL TO ALBERTA | NAIT

Authentication Is Not Just A Form

- login inputs are only one visible part
- the real structure also includes:
  - shared auth state
  - conditional navigation
  - protected backend requests

Important rule

- auth is a state-plus-navigation problem first

WE ARE ESSENTIAL TO ALBERTA | NAIT

Public Routes Versus Protected Routes

Public routes

- welcome screen
- login screen
- register screen

Protected routes

- existing study-plan feature screens
- any route that should only work after sign-in

WE ARE ESSENTIAL TO ALBERTA | NAIT

One Flow To Watch

1. user reaches a public auth screen
2. user registers or logs in
3. backend returns a token and safe user object
4. app stores that result in shared auth state
5. app redirects into the protected route group
6. `/auth/me` validates the current user session
7. protected data request uses the same token pattern

WE ARE ESSENTIAL TO ALBERTA | NAIT

Two Unauthenticated Entry Points

Normal launch

- public welcome screen

Signed-out user tries a protected route

- login screen redirect

Important point

- both routes are public
- they serve different moments in the flow

WE ARE ESSENTIAL TO ALBERTA | NAIT

What Logout Must Do

- remove the token from auth state
- remove the signed-in user from auth state
- block the protected route group again
- send the app back to a public auth route

Not enough

- logout is not only “hide one button”

WE ARE ESSENTIAL TO ALBERTA | NAIT

Where Auth State Should Live

- auth state is not local to one screen
- multiple routes need it at the same time
- the app needs one shared source of truth

Typical lesson-19 values

- current token
- current signed-in user
- auth actions such as sign in and sign out

WE ARE ESSENTIAL TO ALBERTA | NAIT

What Should Not Move Into Auth State

- every temporary text input
- every selected study step
- every form error in the whole app

Keep the boundary clear

- auth state controls access
- feature state controls feature data after access is granted

WE ARE ESSENTIAL TO ALBERTA | NAIT

Protected Route Structure

- Expo Router route groups make the split visible
- one group can hold public auth routes
- one group can hold protected feature routes

Concept clue

- if a signed-out user should not reach it, the protected layout should stop that path

WE ARE ESSENTIAL TO ALBERTA | NAIT

Bearer Token Pattern

Protected requests send:

```
Authorization: Bearer <token>
```

Meaning

- the app is presenting proof of access with that request

WE ARE ESSENTIAL TO ALBERTA | NAIT

Session Validation Example

Preferred lesson-19 example

- `GET /auth/me`

What it confirms

- the token still identifies a valid current user
- the app can trust the current auth session enough to enter protected work

WE ARE ESSENTIAL TO ALBERTA | NAIT

Code Clue To Inspect

- the token is returned by login or registration
- shared auth state keeps the token available
- a request helper attaches that token to:
  - `GET /auth/me`
  - `GET /study-steps`

The question to ask

- where does the token come from before this request runs?

WE ARE ESSENTIAL TO ALBERTA | NAIT

Why `/auth/me` Matters

- it is a protected route
- the backend uses the token to identify the current user
- it gives this lesson one concrete session-validation example
- it proves that auth affects requests, not only screen labels

Lesson boundary

- today: explain the flow
- lesson 20: build the flow

WE ARE ESSENTIAL TO ALBERTA | NAIT

Midpoint Check

By the middle of class, students should know:

- one public route
- one protected route
- one shared auth value
- one explanation of what `/auth/me` validates
- one file that attaches the token to a protected request

WE ARE ESSENTIAL TO ALBERTA | NAIT

Checkpoint

Students should be ready to explain

- why auth needs shared state
- how navigation changes after login
- why logout changes route access
- what `/auth/me` validates in this lesson
- why a protected request needs a bearer token

Status language

- `resolved` : you can explain the full auth flow clearly
- `partial` : one key piece is still fuzzy
- `blocked` : the route split or token flow is still unclear

WE ARE ESSENTIAL TO ALBERTA | NAIT

What We Learned Today

- authentication changes state, navigation, and backend requests
- public and protected routes have different jobs
- auth state should live in one shared place
- bearer tokens allow protected requests after login
- the existing feature app can be patched with auth instead of replaced

WE ARE ESSENTIAL TO ALBERTA | NAIT

Next Step

- lesson 20 will implement:
  - registration and login actions
  - shared auth state
  - protected route gating
  - one working protected request

Ready after lesson 19 means

- `resolved` : you can describe the target build before coding
- `partial` : you need one more file comparison or flow explanation
- `blocked` : you still need help naming the public/protected split

WE ARE ESSENTIAL TO ALBERTA | NAIT